CAI Assignment 2

# Development Evaluation Report

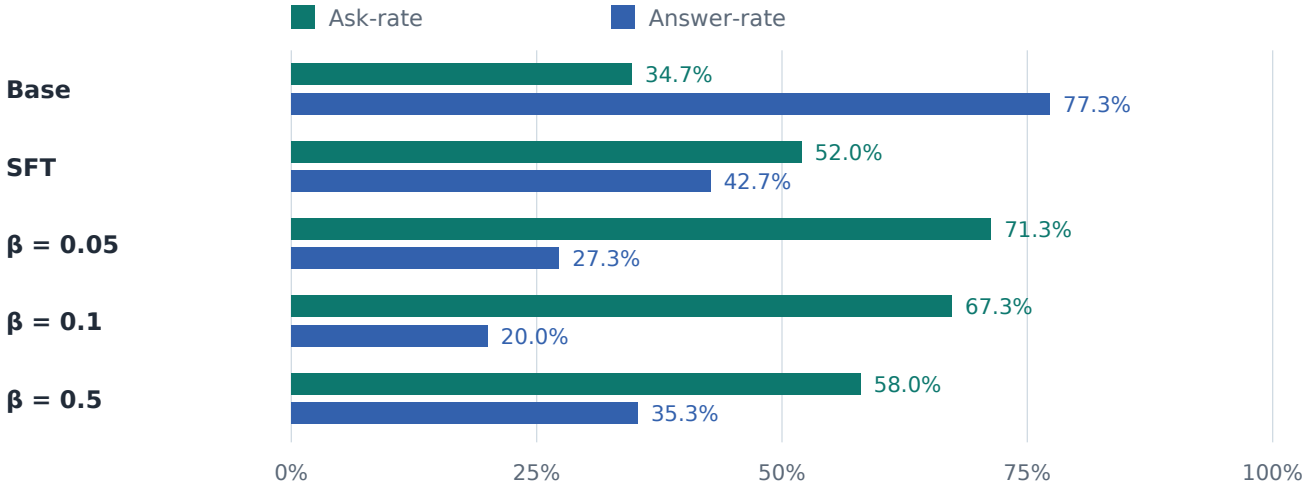
## 1. Evaluation results

You evaluated 150 ambiguous and 150 clear development prompts using the course's Llama 3.1-8B judge.

| Model | Ask-rate ↑ | Answer-rate ↑ | MMLU |
|---|---|---|---|
| **Base** | 34.7% | 77.3% | Not tested |
| **SFT** | 52.0% | 42.7% | Not tested |
| **DPO β = 0.05** | 71.3% | 27.3% | Not tested |
| **DPO β = 0.1** | 67.3% | 20.0% | Not tested |
| **DPO β = 0.5** | 58.0% | 35.3% | Not tested |

### Clarification vs direct-answer performance

Higher is better for both metrics. These results are from your development sets, not the official hidden test.



## 2. Comparison against the grading targets

The README requires at least a 25-percentage-point improvement in ambiguous ask-rate, while clear answer-rate must decrease by no more than 5 percentage points.

| Model | Ask-rate change vs base | Answer-rate change vs base |
|---|---|---|
| **SFT** | +17.3 pp | −34.7 pp |
| **DPO β = 0.05** | +36.7 pp | −50.0 pp |
| **DPO β = 0.1** | +32.7 pp | −57.3 pp |
| **DPO β = 0.5** | +23.3 pp | −42.0 pp |

pp = percentage points.

The β = 0.05 and β = 0.1 runs meet the clarification-improvement target on this development set. None meet the answer-retention target. β = 0.5 has the highest answer-rate among the DPO runs, but still suffers a substantial decline.

These numbers do not establish your official grade, which uses separate hidden evaluation prompts.

## 3. Why the answer-rate is collapsing

The most important finding is the large number of responses labeled other, rather than simply too many clarification questions.

| Model | Clear: ask | Clear: answer | Clear: other |
|---|---|---|---|
| **Base** | 11 | 116 | 23 |
| **SFT** | 15 | 64 | 71 |
| **DPO β = 0.05** | 40 | 41 | 69 |
| **DPO β = 0.1** | 47 | 30 | 73 |
| **DPO β = 0.5** | 25 | 53 | 72 |

Notice that after SFT, other increases from 23 to 71 out of 150 clear prompts. That happens before DPO training.

The judge uses other for responses that are empty, incoherent, refusals, or neither a clear answer nor a clarification question.

This suggests two separate problems worth investigating:

- Behavioral overfitting: The model may have learned very short, repetitive response patterns from your template-generated data.
- Generation or evaluation issues: Some responses may be malformed or generated incorrectly during batched inference.

We cannot determine which is responsible from aggregate scores alone. We need to inspect actual model responses.

## 4. Why results.json failed validation

This warning is separate from your behavioral results:

```
ValidationError: None is not of type 'number'
On instance['runs']['sft']['mmlu_acc']: None
```

Your MMLU file is missing, so the evaluator records:

```
"mmlu_acc": null
```

The schema requires a numerical MMLU accuracy and a nonempty evaluation set. Because your MMLU set has zero examples, the development results.json does not validate.

This does not mean your SFT or DPO checkpoints failed. You can use the behavioral results for debugging, but you will need a properly sourced public MMLU development slice to produce a complete, schema-valid evaluation locally. Don’t replace missing scores with invented numbers.

## 5. My recommended next step: Check a possible padding issue

I inspected your repository’s evaluate.py and utils/model_utils.py.

The evaluator generates responses in batches of 8, but load_tokenizer() does not explicitly set padding_side. That is worth checking because decoder-only Llama models should use left padding for batched generation. Right padding can produce incorrect continuations. Hugging Face documents this behavior.

First, run:

```
python3 - <<'PY'
from utils.model_utils import load_tokenizer
from configs.paths_config import SFT_CHECKPOINT_DIR

tokenizer = load_tokenizer(SFT_CHECKPOINT_DIR)
print("Padding side:", tokenizer.padding_side)
PY
```

If it prints right, test whether that is affecting your model’s output before making permanent changes.

You can compare SFT responses using both settings:

```
python3 - <<'PY'
from evaluate import load_adapter_policy, generate_responses
from data_loader import load_clear_set
from utils.model_utils import load_tokenizer
from configs.paths_config import SFT_CHECKPOINT_DIR

tokenizer = load_tokenizer(SFT_CHECKPOINT_DIR)
model = load_adapter_policy(SFT_CHECKPOINT_DIR)
model.eval()

prompts = [x["prompt"] for x in load_clear_set()[:8]]

for side in ["right", "left"]:
    tokenizer.padding_side = side
    print(f"\n=== Padding: {side} ===")
    responses = generate_responses(model, tokenizer, prompts)
    for i, response in enumerate(responses, 1):
        print(f"{i}. {response!r}")
PY
```

**Run this on your allocated GPU node.**

If left padding produces much more coherent answers, that points to an evaluation-generation issue. A fix should preserve the course's judge and scoring rules, and you should document it if you change the supplied code.

## 6. What to prioritize

1. Check the tokenizer padding and compare actual generated responses.
2. Investigate whether the high other counts represent malformed generations or genuine model behavior.
3. Add an independent public MMLU development slice to measure capability retention and resolve the schema warning.
4. Only consider SFT/DPO retraining if the corrected evaluation still shows major degradation.

Do not choose β = 0.05 simply because it has the highest ask-rate. Your assignment explicitly requires answering clear requests well, and that requirement is currently failing for all fine-tuned models.

The most useful next result is the output of the padding-side check and a few actual SFT responses. That will help identify whether the problem is in evaluation or model training before you spend resources retraining.